SmartTaskPlanner

Architecture Notes

A companion document explaining the reasoning behind each major design decision in the project.

Tech Stack: ASP.NET Core 8 | Angular 18 | C# 12 | TypeScript 5.5

Nemetschek / Spacewell Technical Assessment

Table of Contents

1. Why Clean Architecture?

The solution is split into four projects with a strict dependency direction:

```
API -> Application -> Domain
^
Infrastructure -----------+ (implements Application interfaces)
```

Domain has zero external dependencies. Application depends only on Domain. Infrastructure implements the interfaces defined in Application, not the other way around. API is the thin HTTP shell on top.

The most complex logic - cycle detection, topological sorting, business rules like 'Bug tasks are always High priority' - lives in Domain and Application. Because those layers have no infrastructure dependencies, they can be tested in pure unit tests with no mock HTTP server, no database, and no Angular wiring. The GraphService test spins up the service with a NullLogger and a plain list of TaskItem objects. It runs in milliseconds.

Trade-off

More .csproj files and more interfaces than a simple MVC app. For a small assignment this adds boilerplate, but the payoff is clear: I can swap InMemoryTaskRepository for an EfCoreTaskRepository without touching a single line of TaskService or GraphService.

2. Domain Layer - The Immutable Aggregate

```
public class TaskItem
{
public string Id { get; private set; }
public Priority Priority { get; private set; }
public List<string> Dependencies { get; private set; }
// ...

public void Update(string title, ...) { /* controlled mutation */ }
internal void SetPriority(Priority p) => Priority = p;
}
```

All properties use private set so that no external code can bypass business rules by writing task.Priority = Priority.Low directly. The only legal mutation paths are:

Method	Called by	Purpose
constructor(...)	TaskFactory.Create()	First-time creation
Update(...)	TaskService.UpdateTaskAsync()	PUT request mutation
AssignId(id)	InMemoryTaskRepository.AddAsync()	ID assignment after persistence
SetPriority, SetCategory, SetEstimatedEffort	TaskFactory.ApplyBusinessRules()	Business rule overrides

The rule-override setters (SetPriority, SetCategory, SetEstimatedEffort) use the internal access modifier. This means they are only visible within the Domain assembly. Application, Infrastructure, and API code cannot call them. Only TaskFactory, which lives in Domain, can forcibly override those fields. This is an assembly-enforced contract.

3. Factory Pattern - Enforcing Business Rules at the Seam

A TaskFactory class centralises all type-specific business rules:

- Bug tasks are always set to High priority, regardless of what the user submits.
- Testing tasks default to the 'Quality Assurance' category when no category is provided.
- Development tasks have a minimum estimated effort of 1.

Key design choice: rules run twice

ApplyBusinessRules() is called both in TaskFactory.Create() during creation, and again in TaskService.UpdateTaskAsync() after the entity is mutated by Update(). This means a user cannot create a Bug task with Low priority, and they also cannot update a Bug task to Low priority. The rule is impossible to bypass regardless of which HTTP verb is used.

Why not validate in the DTO validator?

FluentValidation handles structural correctness: Is the title present? Is the effort greater than zero? Domain rules are semantically different - they represent type-specific business logic, not input validation. Mixing them would couple the validator to domain knowledge it should not own.

4. Graph Service - Two Algorithms for One Problem

The GraphService uses two different graph algorithms for two different purposes.

Algorithm 1: DFS with Recursion Stack (Pre-mutation Validation)

Before every create or update operation, a DFS-based cycle detection runs against the full task graph including the proposed new/updated task. I chose DFS here because it naturally produces the cycle path - the exact sequence of task IDs that form the loop. This path is returned to the client in the API response so the frontend can display a human-readable message like 'Task A -> Task B -> Task A'.

The DFS uses two separate sets - visited and recursionStack. The visited set tracks nodes that have been fully explored. The recursionStack tracks nodes on the current path being explored. Without this distinction, the algorithm would incorrectly report a cycle when two independent tasks both point to the same dependency (a diamond shape).

In addition to cycle detection, the validation also checks for self-dependencies (a task depending on itself) and missing dependencies (referencing a task ID that does not exist in the system).

Algorithm 2: Kahn's Algorithm with Priority Queue (Plan Generation)

For the execution plan endpoint (GET /api/tasks/plan), I use Kahn's algorithm - a BFS-based topological sort. It works by iteratively removing nodes with zero in-degree (no remaining dependencies). The key upgrade over plain BFS is replacing the standard FIFO queue with a PriorityQueue using a custom comparer.

At any point where multiple tasks are 'ready' (in-degree = 0), the priority queue ensures the most important one is dequeued first, based on these tie-breaking rules:

Rule	Direction	Rationale
1. Priority (High to Low)	Descending	Critical work first
2. EstimatedEffort (low to high)	Ascending	Quick wins first
3. Task ID (alphabetical)	Ascending	Deterministic output

The third tie-breaker (alphabetical ID) is important for determinism - running the same plan twice on the same data always produces identical output.

Dual Cycle Guard - Defence in Depth

The DFS pre-validation prevents cycles from ever being persisted. But Kahn's algorithm also has a built-in cycle check: if the execution plan contains fewer tasks than the total count, some tasks were never dequeued, meaning they are stuck in a cycle. I kept both checks as a safety net - if somehow a cycle made it past the DFS (e.g., through manual data corruption), plan generation would catch it and throw rather than silently returning an incomplete plan.

5. Repository Contract - The Infrastructure Boundary

```
public interface ITaskRepository
{
Task<IEnumerable<TaskItem>> GetAllAsync(CancellationToken ct);
Task<IEnumerable<TaskItem>> GetPagedAsync(string? cursor, int pageSize,
string? search, CancellationToken ct);
Task<TaskItem?> GetByIdAsync(string id, CancellationToken ct);
```

```
Task<TaskItem> AddAsync(TaskItem task, CancellationToken ct);
Task UpdateAsync(TaskItem task, CancellationToken ct);
Task DeleteAsync(string id, CancellationToken ct);
}
```

The interface is defined in the Application layer. The implementation (InMemoryTaskRepository) lives in Infrastructure. This means Application code depends on the abstraction, not on the concrete store.

Why async when it is in-memory?

The in-memory implementation wraps everything in Task.FromResult(), but the interface uses async Task<> return types. This is intentional. If I were to swap to Entity Framework or Dapper, the interface and all call sites would remain unchanged. Only the Infrastructure layer would change.

Why Singleton, not Scoped?

InMemoryTaskRepository is registered as a Singleton in DI. A Scoped registration would create a new (empty) ConcurrentDictionary instance per HTTP request, wiping all data on every call. The ConcurrentDictionary handles thread safety for concurrent reads and writes without explicit locking.

6. Keyset Pagination - Why Not Offset?

Problems with offset pagination (SKIP N, TAKE M)

- Data drift: If a task is inserted between page 1 and page 2, items shift. Some are skipped, others duplicated.

- O(N) scan: Deep pages get progressively slower because the store must scan and discard all prior rows.

How the keyset cursor works

Tasks are sorted by CreatedAt ascending, then by Id ascending (for a fully unique ordering). The cursor is a Base64-encoded string containing both values:

```
Cursor = Base64( CreatedAt.ToString("O") + "|" + Id )
Decoded example: 2026-07-02T19:50:38.1234567Z|D-101
```

The next page is fetched by filtering to rows where CreatedAt > cursorCreatedAt, or where CreatedAt equals cursorCreatedAt and Id > cursorId. This gives O(1) position lookup with no rows scanned before the cursor.

Base64 encoding makes the cursor opaque to the client. A future schema change could evolve the cursor format without breaking clients that treat it as a black box. If the cursor is malformed or stale, the implementation silently falls back to page 1 instead of crashing the user's session.

7. Error Handling Pipeline - Typed Exceptions to RFC 7807

All domain exceptions are caught in a single ExceptionHandlingMiddleware and translated to RFC 7807 ProblemDetails responses. Controllers contain zero try/catch logic - they stay thin and focused on HTTP-level concerns.

Exception	HTTP Status	Rationale
TaskNotFoundException	404 Not Found	Resource does not exist
CircularDependencyException	409 Conflict	Request conflicts with graph state
SelfDependencyException	400 Bad Request	Structurally invalid input
DependencyNotFoundException	400 Bad Request	Referenced task ID missing
DomainException (base)	400 Bad Request	Generic domain rule violated
Everything else	500 Internal Server Error	Unexpected failure

For CircularDependencyException, the ProblemDetails response includes an extensions field called cyclePath containing the array of task IDs in the cycle. This is what the Angular CycleDialogComponent reads to display the human-readable cycle path and offer the 'force' bypass option.

8. Force Flag - Intentional Rule Bypass

Both POST /api/tasks and PUT /api/tasks/{id} accept a ?force=true query parameter. In the real world, project managers sometimes need to create tasks with dependency relationships that are known to be temporarily cyclical. The force flag allows this.

Validation Check	Without force	With force=true
Title required	Blocked	Blocked
EstimatedEffort > 0	Blocked	Blocked
Dependency must exist	Blocked	Blocked
Self-dependency	Blocked	Blocked
Circular dependency	Blocked	Allowed

The force flag is scoped only to CircularDependencyException. Allowing a malformed task through (bad title, missing dependency ID) would create corrupted data with no recovery path. A circular dependency, by contrast, can be resolved by the user later.

9. Clone-on-Validate Pattern for Updates

When updating a task, the service does not mutate the existing in-memory entity first and then validate. Instead, it creates a temporary TaskItem clone with the proposed new values, runs ApplyBusinessRules and ValidateGraph against that clone, and only if everything passes does it call Update() on the real entity.

This prevents partial state corruption. If validation fails (for example, the new dependencies would introduce a cycle), the in-memory entity is untouched. The user gets an error response and can retry. Without this pattern, a failed validation would leave the entity in a half-updated state because Update() would have already been called.

10. ID Generation Scheme

IDs are generated server-side in the repository using a type-prefixed format:

Task Type	Prefix	Starting Number	Example
Development	D-	101	D-101, D-102, D-103
Testing	T-	201	T-201, T-202
Bug	B-	301	B-301, B-302
General	G-	401	G-401, G-402

The prefix makes task type immediately visible in logs, UI, and API responses without needing to look up the full object. The repository scans existing IDs with the same prefix and assigns Max + 1. Clients never provide IDs.

11. DI Wiring and Service Lifetimes

Each layer has its own DependencyInjection.cs extension method so that Program.cs stays clean:

```
// Program.cs
builder.Services.AddApplication(); // registers services, factory, validators
builder.Services.AddInfrastructure(); // registers repository
```

Service lifetimes are chosen deliberately:

Service	Lifetime	Why
TaskService	Scoped	Stateless orchestrator - one instance per request is fine
GraphService	Scoped	Stateless computation - no shared state between requests
TaskFactory	Scoped	Stateless factory - safe per request
InMemoryTaskRepository	Singleton	Must persist data across all requests
FluentValidation validators	Auto-registered	Discovered by assembly scanning

The key insight is that the repository is a Singleton while services are Scoped. This works because ConcurrentDictionary is thread-safe. A Scoped repository would lose all data between requests.

12. DTOs as Immutable Records

```
public record CreateTaskDto(
string Title,
string Description,
Priority Priority,
int EstimatedEffort,
string Category,
TaskType Type,
List<string> Dependencies);
```

All DTOs (CreateTaskDto, UpdateTaskDto, TaskResponseDto, PagedResponseDto, TaskLookupDto) are defined as C# records rather than classes. Records are immutable by default, provide value-based equality, and keep the DTO definitions concise. CreateTaskDto intentionally excludes Id and Status (server assigns these), while UpdateTaskDto includes Status so the user can change it on update.

The PagedResponseDto is generic, wrapping any item type with Items, NextCursor, and HasNext fields. This makes it reusable if other paginated endpoints are added later.

13. Angular State Architecture

```
TaskService (Singleton, root-level)
paginatedTasksSubject: BehaviorSubject<TaskItem[]> -- page accumulation
errorSubject: BehaviorSubject<string|null> -- global error
nextCursor: string | null
hasNextPage: boolean
isPageLoading: boolean
isInitialLoading: boolean
```

The task list page uses a 'Load More' infinite scroll pattern. If the cursor and accumulated task list lived inside the component, navigating away and back would reset everything. By holding state in the singleton

TaskService, the accumulated list survives navigation.

Replace vs. Append asymmetry

loadInitialPage() replaces the BehaviorSubject value entirely (fresh search or filter). loadNextPage() appends to it (infinite scroll 'load more'). Search resets context. Pagination extends it. This maps directly to user expectations.

Local cache for task detail

When a user clicks a task to view its detail, the getTask() method first checks the paginated task list in memory. If the task is already there, it returns immediately with zero HTTP round-trips. A forceRefresh parameter lets components bypass the cache when they need fresh data from the server.

ApiService abstraction

All HTTP calls go through a single ApiService that centralises error handling. It parses ProblemDetails from the server, maps errors to human-readable messages, and passes 409 Conflict responses upstream unmodified so that the cycle dialog can read the cyclePath extension field.

14. Reactive Patterns in Angular

Search debouncing

The search input is wired to a BehaviorSubject. debounceTime(500) waits 500ms after the user stops typing before triggering an API call. distinctUntilChanged() prevents a redundant call if the user types and then deletes the same characters.

Subscription leak prevention

The task form uses takeUntilDestroyed(this.destroyRef) - Angular 16+'s idiomatic way to automatically complete a subscription when the component is destroyed. This replaces the older Subject + takeUntil + ngOnDestroy pattern and prevents memory leaks when users navigate away before an HTTP call resolves.

Cycle dialog as an Observable

The CycleDialogService.open() returns an Observable rather than using a callback. The calling code subscribes and decides what to do based on the result. This is the RxJS-idiomatic way to model 'wait for user interaction' as a stream.

Confirmation modal

The delete flow uses a ConfirmModalService to show a confirmation dialog instead of the browser's native window.confirm(). This keeps the UI consistent and allows styling the modal to match the rest of the application.

15. Cross-Cutting Concerns

Structured logging with Serilog

Every service method uses structured log parameters ({TaskId}, {Count}, {Cursor}) rather than string interpolation. This enables log aggregation tools like Seq or Elasticsearch to filter and query by specific fields. Serilog is configured via appsettings.json so log levels and sinks can change per environment without recompiling.

CORS via configuration

Allowed origins are read from the AllowedOrigins section in appsettings.json rather than hardcoded. In production, this would point to the deployed Angular URL. In development, it allows http://localhost:4200.

CancellationToken propagation

Every async method in the stack - controller, service, repository - accepts and forwards a CancellationToken. If the HTTP client disconnects mid-request, ASP.NET Core signals cancellation and the operation stops cleanly.

FluentValidation auto-registration

Validators are discovered by assembly scanning (AddValidatorsFromAssemblyContaining). ASP.NET Core automatically runs them on bound DTOs before the controller action body executes. Invalid input is rejected with a 400 response before the service layer is even reached.

Delete protection

A task cannot be deleted if other tasks depend on it. The service checks all existing tasks for references to the target ID and throws a DomainException if any are found. Users must remove or update the dependents first.

Seed data

The InMemoryTaskRepository constructor populates 11 realistic seed tasks that model a real software project: project setup, authentication, dashboard UI, testing, deployment, a bug fix, and documentation. These tasks have real interdependencies forming a DAG, making it easy to test the execution plan without manually creating test data.

16. What Was Left Out and Why

These are deliberate omissions, not oversights.

What was omitted	Why
Authentication / Authorization	The assignment focuses on task management logic. JWT middleware would add ~200 lines that obscure the core design being evaluated.
Database / EF Core	The repository interface is designed so swapping to EF Core only changes the Infrastructure layer. Application and Domain are completely unaffected.
IQueryable from repository	Exposing IQueryable would leak infrastructure concerns (LINQ-to-SQL) into Application. All filtering is done in-memory, which is acceptable for this scope.
Frontend unit tests	This is the most significant gap. ~90 backend test cases exist but no Angular spec files. TaskService and TaskFormComponent would be the highest-value targets.
Parallel execution waves	The plan returns a totally ordered list. Grouping tasks into parallel waves is a natural extension of Kahn's algorithm but was left out to keep the response format simple.
Swagger XML comments	Controller actions would benefit from XML doc comments for richer Swagger UI output. Currently only the domain layer has documentation comments.

These notes explain the why behind each design choice, not just the what. They are a companion to the README.md.